

Final Project Report

AI System Building with GenAI as a Cognitive Partner

AI 100 · Heart Disease Classification on Tabular Data

I. What I Learned About GenAI

1.1 GenAI as a Socratic Tutor, Not an Answer Machine

The most striking insight from this project is that GenAI is most educationally valuable when it withholds direct answers. When I submitted a weak reflection for Bug 1 (setting `test_size=1.5`), the model responded with 'Bad' and asked me to think about the two distinct interpretations of the `test_size` parameter — float vs integer — rather than simply telling me the fix. This forced me to revisit the sklearn documentation myself, which cemented the understanding far more deeply than if the solution had been handed to me.

Similarly, for Bug 7 (removing `random_state`), I initially dismissed the non-reproducibility as 'small and acceptable variance.' The model's Socratic guidance pushed me to think about reproducibility as a scientific principle, not just a numerical concern — asking what would happen if two collaborators ran the same code and got different results, and how one would report or debug such findings. This reframing was more valuable than any code fix.

1.2 GenAI Accurately Distinguishes Deep vs. Surface Reasoning

Of the 10 cases, 4 received 'Bad' labels (Cases 1, 2, 7, 10) and 6 received 'Good' labels. Looking back, the pattern is clear: my 'Bad' reflections named the symptom ('1.5 is too large') without reasoning about mechanism, while my 'Good' reflections identified the causal chain ('`X_train_s` has 400 rows because 20% was split out, but `y` still has 500 rows'). GenAI was remarkably reliable at distinguishing these.

The most instructive 'Bad' case was Bug 10 (swapped `accuracy_score` arguments). I concluded the swap was 'fine' because the number was the same. The model's response exposed a gap in my reasoning: I had not considered that `classification_report` is asymmetric — swapping `y_true` and `y_pred` changes which array defines the 'ground truth,' inverting the semantic meaning of precision and recall for each class. A numerically identical report can be analytically incorrect.

1.3 GenAI Has Consistent Blind Spots

GenAI tended to give 'Good' even when my reflection was technically correct but perhaps not fully deep. For instance, Bug 5 (calling `predict` before `fit`) was labeled Good with a fairly surface-level

reflection. This suggests GenAI is better at detecting wrong reasoning than at pushing correct but shallow reasoning to a higher level of insight. For maximum learning value, students should voluntarily deepen their reflections even after receiving 'Good.'

Example of a GenAI guidance response that was particularly helpful (Bug 2 — data leakage):

"fit_transform re-learns mean and std from the test set... Why does a StandardScaler need to be fitted at all? If you fit the scaler separately on the test set, what statistics is it now using — and should those statistics come from the test set?"

This single response taught me the conceptual distinction between fitting (learning parameters from data) and transforming (applying learned parameters), and why mixing the two across train/test boundaries constitutes information leakage.

II. What I Learned About Building AI Systems

2.1 Silent Bugs Are More Dangerous Than Crashes

Four of my ten bugs produced no error at all (Cases 2, 7, 10, and a variant of Case 3). In a production pipeline, any of these would silently corrupt model evaluation. Bug 2 (data leakage via `fit_transform` on test data) is particularly insidious: accuracy actually improved from 0.87 to 0.89, which a naive developer might interpret as a positive change. This taught me that rising metrics are not always evidence of improvement — they can be evidence of evaluation contamination.

Bug 10 (swapped `classification_report` arguments) is similarly dangerous at scale: a team relying on precision/recall figures to decide model deployment thresholds could make systematically wrong decisions based on inverted metrics. The lesson is that unit tests and API-contract awareness are as important as model performance.

2.2 Reproducibility Is a First-Class Requirement

Bug 7 demonstrated that omitting `random_state` from both `train_test_split` and `RandomForestClassifier` produces different accuracy values across runs. In this particular dataset, variance was empirically small, which made it tempting to dismiss. But reproducibility is not only about variance magnitude — it is about the ability to re-run an experiment and get the same result, which is foundational to scientific validity, collaborative debugging, and fair model comparison.

2.3 Sklearn's API Contract Must Be Respected Precisely

Several bugs arose from misunderstanding sklearn's strict parameter contracts. Bug 4 (`n_estimators` as a string '100' instead of integer 100) and Bug 9 (`max_depth=0`) both produce `InvalidParameterError` with clear messages — but only if you know to read them carefully. Bug 3 and Bug 8 showed that length mismatches between arrays passed to `fit()` or `stratify=` are caught immediately. Bug 6 showed that feature-count mismatches between `fit` and `predict` are caught at inference time. Understanding

where sklearn validates inputs helps localize bugs much faster.

2.4 The Preprocessing Pipeline Must Be Fit Only on Training Data

This is perhaps the most practically important lesson. In Bug 2, re-fitting the StandardScaler on the test set caused the test data to be normalized using its own statistics (mean, std) rather than those of the training set. In a real deployment scenario, incoming inference data would be scaled using training statistics — so evaluation using test-set statistics creates an unrealistic and overly optimistic benchmark. The rule is: fit preprocessors on `X_train` only, then transform both `X_train` and `X_test` using those fitted parameters.

2.5 Intentional Bug Injection Is a Powerful Learning Method

Before this project, I debugged bugs reactively — when they appeared. Intentionally introducing bugs forced me to think prospectively: which changes are likely to break the system, and in what way? This perspective shift is the difference between knowing that `fit_transform` on test data is wrong and understanding why, when it would fail silently, and what evaluation artifact it creates. The deliberate bug-injection methodology makes abstract ML best practices concrete and memorable.

Summary of Bug Cases

Case	Bug Type	Label	Key Lesson
1	<code>test_size=1.5</code>	Bad	Float vs int parameter semantics
2	<code>fit_transform</code> on test set	Bad	Data leakage in preprocessing
3	<code>y</code> vs <code>y_train</code> in <code>fit()</code>	Good	Array length consistency
4	<code>n_estimators</code> as string	Good	Type strictness in sklearn
5	<code>predict</code> before <code>fit</code>	Good	API call ordering
6	Wrong feature count	Good	Feature shape validation at inference
7	No <code>random_state</code>	Bad	Reproducibility principle
8	<code>stratify=y_train</code>	Good	Stratify array must match input length
9	<code>max_depth=0</code>	Good	Valid parameter ranges
10	Swapped metric args	Bad	API asymmetry in evaluation functions